

Бассейн (Piscine)

Основы программирования на Go и работы в командной строке

Модель 01 Edu · peer-to-peer · 8 учебных недель

Форма обучения	Очная, на кампусе; доступ 24/7
Язык обучения	Английский
Продолжительность	8 недель
Минимальная нагрузка	15 часов в неделю, с контролем присутствия
Форматы учебной работы	Quest · Checkpoint · Raid
Недели с чекпоинтами	2, 4, 6, 8 — пятница и суббота, два потока в день
Групповые рейды	Недели 2, 4, 6 — онлайн; защита на следующей неделе
Основные технологии	Go, Bash, Git

1. Модель обучения

Обучение построено на международной платформе 01 Edu по модели peer-to-peer: равный обучает равного. Лекций и преподавателей в привычном смысле нет.

Студент получает техническое задание, разбирается самостоятельно и с сокурсниками, доводит работу до состояния готового продукта и защищает её. Техническое сопровождение обеспечивают менторы: они не читают материал, а помогают распутать затруднение.

Как это работает на практике

- **Задание вместо лекции.** Каждое упражнение описано как техническое задание: что должна делать программа, какие функции ожидаются, какой вывод считается правильным. Способ решения студент находит сам.
- **Точность вывода как критерий.** Проверка автоматическая и буквальная: важен каждый пробел, запятая и перевод строки. Формат вывода сверяют командой `cat -e`, где конец строки виден как `$`.
- **Взаимное обучение.** Разбор чужого решения, объяснение своего и совместная отладка — штатный рабочий процесс, а не помощь «на стороне».
- **Менторская поддержка.** Ментор не выдаёт ответ, а задаёт вопросы, помогающие увидеть ошибку в рассуждении.
- **Прогресс по карте знаний.** Движение идёт по интерактивной карте: следующий блок открывается только после закрытия предыдущего, уровень пропустить нельзя.
- **Дисциплина присутствия.** Не менее 15 часов в неделю на кампусе; доступ к кампусу круглосуточный, ритм студент выстраивает сам.

Что такое бассейн (piscine)

Бассейн — интенсивный погружной этап, в котором кандидат проходит большой объём практики в едином ритме со всей группой. Формат заимствован из методологии 01 Edu, где бассейн укладывают в 26 дней — примерно четыре недели. В настоящей программе тот же объём практики распределён на **8 недель**: недельная нагрузка становится посильнее, а требования к результату, порядок прохождения и способ проверки сохраняются полностью. Принцип, которым бассейн описывают в сети школ, остаётся в силе: кандидаты либо выплывают, либо тонут — темп задан, и удержать его можно только работая.

Бассейн решает две задачи:

1. **Отбор** — показать, кто способен учиться самостоятельно и в сообществе.
2. **Выравнивание** — привести подготовку прошедших к общему уровню перед основным обучением.

2. Три типа заданий и ритм программы

Внутри бассейна кандидат сталкивается с тремя разными форматами работы. Они отличаются и по составу участников, и по способу проверки.

Формат	Участники	Когда и где	Суть и способ проверки
Quest квест	индивидуально	ежедневно на протяжении всех 8 недель, кроме часов чекпоинта; на кампусе	Основной рабочий формат. Тематический блок — связанный набор задач вокруг одной темы, который сдаётся целиком. Следующий квест открывается только после закрытия предыдущего. Прогресс идёт по интерактивной карте знаний, пропустить уровень нельзя.
Checkpoint чекпоинт (экзамен)	индивидуально	недели 2, 4, 6, 8 пятница и суббота, два потока в день; очно, в аудитории	Ближайший аналог экзамена в университетской практике. Кандидат решает задачи по материалу пройденных недель самостоятельно, очно, в ограниченное время. Именно чекпоинт показывает, освоен материал или задачи были закрыты чужими руками.
Raid рейд	группа	выдача — недели 2, 4, 6 ; работа — на выходных (сб и вс), онлайн; защита — на следующей неделе (3, 5, 7), очно перед менторами и преподавателями	Групповой проект. Существует для того, чего индивидуальные задачи не дают: договориться о разделении работы, состыковать чужой код со своим, довести общий результат до защиты в срок.

Общая структура: 8 недель, четыре пары

Недели идут парами. Первая неделя пары — накопление материала: квесты и, начиная с третьей недели, защита рейда, выданного ранее. Вторая неделя пары — проверка: чекпоинт в конце недели и выдача нового рейда, над которым группа работает онлайн на выходных. Таким образом за программу проходит **четыре чекпоинта** (недели 2, 4, 6, 8) и **три рейда** (недели 2, 4, 6) с защитой на неделях 3, 5 и 7.

Неделя	Учебные блоки (ориентировочное распределение)	Checkpoint	Raid
1	Quest 01 — командная строка, Git, shell; Quest 02 — вывод, циклы, условия в Go	—	—
2	Quest 03 — указатели и преобразование типов; Quest 04 — рекурсия и алгоритмы	Чекпоинт № 1 пт и сб	Рейд № 1 — выдача; работа онлайн на выходных
3	Quest 05 — строки, руны, системы счисления	—	Защита рейда № 1 перед менторами
4	Quest 06 — аргументы командной строки; Quest 07 — слайсы и обработка строк	Чекпоинт № 2 пт и сб	Рейд № 2 — выдача; работа онлайн на выходных
5	Блок quad — рисование фигур; Quest 08 — файлы, структуры, обработка ошибок	—	Защита рейда № 2 перед менторами
6	Quest 09 — функции высшего порядка	Чекпоинт № 3 пт и сб	Рейд № 3 — выдача; работа онлайн на выходных
7	Quest 11 — связанные списки	—	Защита рейда № 3 перед менторами
8	Quest 12 — бинарные деревья; повторение и закрытие незакрытых блоков	Финальный чекпоинт № 4 пт и сб	—

Распределение блоков по неделям ориентировочное: темп задаёт карта знаний, и студент может идти быстрее. Обязательным остаётся одно — материал, вошедший в очередной чекпоинт, должен быть закрыт к концу соответствующей недели.

Расписание чекпоинтов

Чекпоинты проходят на неделях 2, 4, 6 и 8 в пятницу и субботу, очно и индивидуально. В каждый из двух дней организуются два потока по 2 часа 30 минут.

День	Поток	Время	Длительность

Пятница	Поток 1	08:00 — 10:30	2 ч 30 мин
	Поток 2	10:30 — 13:00	2 ч 30 мин
Суббота	Поток 1	08:00 — 10:30	2 ч 30 мин
	Поток 2	10:30 — 13:00	2 ч 30 мин

- Каждый студент закреплён за одним днём и одним потоком; распределение объявляется заранее.
- Деление на четыре сессии позволяет проводить экзамен очно при ограниченном числе рабочих мест в аудитории.
- Наборы задач разных потоков различаются по формулировкам, но равны по объёму и уровню сложности и опираются на материал двух прошедших недель.
- Время фиксировано: то, что не сдано к концу потока, не оценивается. Работа ведётся без помощи сокурсников и внешних источников.

Рейды: работа онлайн и защита перед менторами

Рейд выдаётся группе на неделях 2, 4 и 6, а работа над ним ведётся **на выходных — в субботу и воскресенье — онлайн**: участники работают удалённо, самостоятельно договариваясь о разделении задач, ведении общего репозитория и стыковке кода. Учебные будни при этом остаются за квестами. Защита проходит **на следующей неделе** — очно, перед менторами и преподавателями.

На защите группа демонстрирует работающую программу, объясняет устройство решения и отвечает на вопросы. Спросить могут любого участника про любую часть кода, поэтому «разошлись по задачам и не смотрели чужое» не проходит.

Что оценивается	Как это проверяется
Работоспособность решения	Запуск программы на примерах из задания и на дополнительных случаях от менторов
Соответствие ТЗ и формату вывода	Буквальная сверка вывода, включая пробелы и переводы строк
Понимание решения каждым	Вопросы к любому участнику по любой части кода
Организация работы группы	История коммитов, разделение задач, способ стыковки частей
Соблюдение срока	Готовность к защите в назначенное время следующей недели

Недельный ритм

Тип недели	Пн — Чт	Пятница	Суббота	Воскресенье
Без экзамена недели 1, 3, 5, 7	Квесты	Квесты	Квесты; на неделях 3, 5, 7 — защита рейда	Квесты
С экзаменом недели 2, 4, 6, 8	Квесты	Чекпоинт : поток 1 — 08:00-10:30, поток 2 — 10:30-13:00; далее квесты	Чекпоинт (второй день, те же потоки); выдача рейда, далее рейд онлайн	Рейд онлайн (недели 2, 4, 6)

В неделях без экзамена всё время отдано квестам: студент продвигается по карте знаний и, если нужно, закрывает долги по предыдущим блокам. Точные дата и время защиты рейда объявляются менторами в начале недели.

Логика последовательности

Материал выстроен так, что каждый следующий блок опирается на предыдущий: сначала командная строка и Git как рабочая среда, затем базовый вывод и управляющие конструкции Go, затем указатели и работа с памятью, рекурсия и алгоритмы, строки, аргументы командной строки, слайсы, файлы и структуры, функции высшего порядка и, наконец, динамические структуры данных — связные списки и бинарные деревья.

3. Карта программы

Ниже — сводка тематических блоков бассейна. Нумерация приведена по материалам курса.

Блок	Тема	Ключевые навыки
Quest 01	Командная строка, Git, shell-скрипты	Репозиторий, клонирование, коммит; <code>ls</code> , <code>find</code> , <code>sort</code> , <code>tar</code> , права доступа, символические ссылки; <code>curl</code> + <code>jq</code>
Quest 02	Первые программы на Go: вывод, циклы, условия	Печать символов, последовательности, комбинации цифр, форматирование вывода
Quest 03	Указатели, память, строки и преобразование типов	Многоуровневые указатели, обмен значений, длина и разворот строки, <code>Atoi</code> , сортировка слайса
Quest 04	Итерация, рекурсия, алгоритмы	Факториал, степень, Фибоначчи, простые числа, квадратный корень, восемь ферзей
Quest 05	Строки и руны	Индексация, сравнение, поиск подстроки, конкатенация, регистр, валидация, системы счисления
Quest 06	Аргументы командной строки	Чтение и обработка аргументов, реверс, сортировка по ASCII, флаги, обработка ошибок
Quest 07	Слайсы и обработка строк	Генерация диапазонов, конкатенация, разбиение по разделителю, конвертация систем счисления
quad	Графический вывод: рисование прямоугольников	Условия по границам фигуры, повторяющийся вывод, точное соответствие образцу — база для рейда <code>quadchecker</code>
Quest 08	Файлы, структуры, обработка ошибок	Чтение файлов, стандартный ввод, реализация аналогов <code>cat</code> и <code>tail</code> , структуры и указатели
Quest 09	Функции высшего порядка	<code>ForEach</code> , <code>Map</code> , <code>Any</code> , <code>CountIf</code> , <code>IsSorted</code> , сортировка с пользовательским компаратором, программа <code>doop</code>
Quest 11	Связные списки	Вставка, удаление, размер, обход, разворот, поиск, слияние и сортировка списка
Quest 12	Бинарные деревья	Вставка, обходы (in-order, preorder, postorder, по уровням), поиск, минимум/максимум, высота, удаление узла

Чем заканчивается прохождение

К концу бассейна студент умеет самостоятельно поставить себе рабочую среду, разобрать незнакомое техническое задание, написать и отладить программу на Go со строгими требованиями к выводу, работать с динамическими структурами данных и довести групповой проект до защиты.

4. Программа по квестам

Quest 01 — Командная строка, Git и shell-скрипты · 17 упражнений

Стартовый блок задаёт рабочую среду: студент учится создавать и использовать Git-репозиторий, клонировать его и загружать файлы, писать shell-скрипты, управлять файлами и правами доступа, получать и обрабатывать JSON-данные.

Что делает студент

- **introduction** — создаёт репозиторий на Gitea, клонирует его, пишет скрипт `hello.sh`, который печатает приветствие с именем пользователя, и отправляет работу на проверку через Git.
- **make-it-better** — создаёт файлы и каталоги так, чтобы вывод `ls` в точности совпал с образцом: выставляет права, делает символические ссылки, упаковывает результат в архив `tar`.
- **who-are-you, to-git-or-not-to-git, cl-camp4** — получают JSON по URL через `curl` и разбирают его через `jq`: находят запись по идентификатору и выводят нужные поля (имя, сила, пол, состав семьи героя из переменной окружения `HERO_ID`).
- **cl-camp1** (`mastertheLS`) — вывод содержимого каталога без скрытых файлов, через запятую, с сортировкой по времени доступа.
- **cl-camp2, cl-camp7** — точное создание файлов с заданным содержимым и «неудобными» именами со спецсимволами.
- **cl-camp3, cl-camp5** — поиск файлов через `find` по шаблонам имени; вывод `.sh`-файлов без расширения в обратном порядке (`find`, `sort`, `cut`).
- **cl-camp6** (`countfiles.sh`) — подсчёт числа обычных файлов и каталогов в текущем каталоге и подкаталогах.
- **cl-camp8** (`skip.sh`) — вывод `ls -l` через `awk` или `sed` с пропуском каждой второй строки.
- **now-get-to-work, explain, teacher** — сквозной сценарий работы с набором данных: извлечение фрагментов текста через `head` и `tail`, вывод сведений о свидетеле и номере допроса, списка подозреваемых по алфавиту, работа с переменными окружения (`MAIN_SUSPECT`).

Осваивается: Git · Bash · права доступа и ссылки · tar · curl · jq · find/sort/cut/awk/sed · точность формата вывода

Quest 02 — Вывод, циклы и условия в Go · 8 упражнений

Первый блок на Go. Студент отрабатывает печать символов и управление форматом вывода, работу с последовательностями, циклы, условия и создание функций.

Что делает студент

- **printalphabet, printreversealphabet, printdigits** — выводит латинский алфавит в прямом и обратном порядке и цифры 0–9 одной строкой, без лишних символов и без приведения типов.
- **isnegative** — функция `IsNegative`: печатает `T` для отрицательного числа и `F` в остальных случаях.
- **printcomb** — все уникальные комбинации трёх различных цифр по возрастанию, через запятую с пробелом.
- **printcomb2** — все комбинации двух различных двузначных чисел в возрастающем порядке с заданным форматированием.
- **printnbr** — печать значений типа `int` без преобразования в `int64`: отрицательные, ноль, положительные.
- **printcombn** — комбинации `n` различных цифр; требование к эффективности, решение не должно уходить в таймаут.

Осваивается: вывод символов · циклы · условия · объявление функций · форматирование · эффективность кода

Quest 03 — Указатели, строки и преобразование типов · 13 упражнений

Блок о работе с памятью. Студент учится менять значения переменных через указатели, управлять памятью на нескольких уровнях указателей, возвращать несколько значений из функции и преобразовывать строки в числа.

Что делает студент

- **pointone, ultimategointone** — установка значения через указатель и через указатель на указатель на указатель.
- **divmod, ultimatedivmod** — деление с записью частного и остатка по указателям, в том числе с записью результата в исходные переменные.
- **swap** — обмен значений двух целых через указатели.
- **printstr, strlen, strrev** — посимвольная печать строки, подсчёт рун, разворот строки.
- **basicatoi, basicatoi2, atoi** — преобразование строки в число: от простого случая только с цифрами до полноценной обработки знаков `+` и `-` и возврата `0` для некорректного ввода.
- **sortintegertable** — сортировка слайса целых по возрастанию.

Осваивается: указатели · управление памятью · возврат нескольких значений · руны и строки · конвертация типов · сортировка

Quest 04 — Итерация, рекурсия и алгоритмы · 9 упражнений

Блок об алгоритмическом мышлении: итеративные и рекурсивные решения одной задачи, обработка некорректного ввода и переполнения, эффективность.

Что делает студент

- **iterativefactorial, recursivefactorial** — факториал в цикле и через рекурсию; возврат `0` при некорректном вводе или переполнении.
- **iterativepower, recursivepower** — возведение в степень двумя способами; отрицательная степень даёт `0`; рекурсивная версия — без циклов.
- **fibonacci** — рекурсивное вычисление числа Фибоначчи по индексу, `-1` для отрицательного индекса, `for` запрещён.
- **sqr** — возвращает целый квадратный корень или `0`, если корень не целый.
- **isprime, findnextprime** — проверка числа на простоту и поиск ближайшего простого не меньше заданного; требуется алгоритм, не уходящий в таймаут.
- **eightqueens** — рекурсивное решение задачи о восьми ферзях с выводом всех допустимых расстановок в заданном формате.

Осваивается: рекурсия · обработка ошибок и переполнения · условная логика · сложность и оптимизация · backtracking

Quest 05 — Строки, руны и системы счисления · 23 упражнения

Самый объёмный блок бассейна: полный набор операций над строками — от получения одного символа до конвертации чисел в произвольной системе счисления.

Что делает студент

- **first rune, last rune, n rune** — получение первой, последней и n-й руны строки, обработка выхода за границы.
- **compare, index, concat** — сравнение строк (0, -1, 1), поиск позиции подстроки (-1, если не найдена), конкатенация.
- **alphacount** — подсчёт букв латинского алфавита, игнорируя цифры, символы и пробелы.
- **isupper, islower, isalpha, isnumeric, isprintable** — набор проверок содержимого строки.
- **toupper, tolower, capitalize** — преобразование регистра, в том числе приведение каждого слова к виду с заглавной первой буквой.
- **printnbrinorder, trimatoi** — вывод цифр числа по возрастанию; извлечение числа из «загрязнённой» строки с учётом знака.
- **basicjoin, join** — склеивание слайса строк без разделителя и с заданным разделителем.
- **printnbrbase, atoi** — печать числа в произвольной системе счисления и обратное преобразование; валидация базы (длина, уникальность символов, отсутствие + и -), вывод NV для некорректной базы.

Осваивается: руны · индексация и сравнение строк · валидация · регистр · соединение строк · системы счисления

Quest 06 — Аргументы командной строки · 7 упражнений

Блок про взаимодействие программы с пользователем: чтение аргументов, их преобразование, флаги и справка.

Что делает студент

- **printprogramname** — программа печатает собственное имя; попутно разбирается компиляция и запуск разными способами.
- **printparams, revparams** — вывод каждого аргумента с новой строки и вывод аргументов в обратном порядке.
- **sortparams** — сортировка аргументов в порядке ASCII.
- **nbrconvertalpha** — перевод чисел в буквы алфавита по позиции (1 → a), пробел для некорректного ввода, флаг --upper для верхнего регистра.
- **flags** — обработка флагов: --insert для вставки строки, --order для сортировки по ASCII, --help и пустой вызов для вывода справки с точным форматированием.
- **rotatevowels** — зеркальная перестановка гласных в аргументах; пустой вызов — перевод строки, отсутствие гласных — вывод без изменений.

Осваивается: os.Args · валидация ввода · флаги и справка · сортировка · обработка строк

Quest 07 — Слайсы и обработка строк · 7 упражнений

Блок о слайсах: их создание с ограничениями по инструментам, разбиение и склеивание строк, конвертация между системами счисления.

Что делает студент

- **appendrange** — слайс целых от `min` включительно до `max` исключая; `nil`, если `min` не меньше `max`; без `make`.
- **makerange** — та же задача, но без `append` — с управлением ёмкостью слайса.
- **concatparams** — склеивание слайса строк через `\n`.
- **splitwhitespaces** — разбиение строки на слова по пробелам, табуляциям и переводам строк.
- **printwordstables** — печать каждого элемента слайса с новой строки.
- **split** — разбиение строки по произвольному разделителю.
- **convertbase** — перевод числа из одной системы счисления в другую с возвратом строкового результата.

Осваивается: слайсы и ёмкость · `append` vs `make` · разбиение и склеивание строк · базы счисления

Блок quad — рисование фигур · 5 упражнений: `quadA-quadE`

Пять функций, печатающих корректный прямоугольник заданной ширины `x` и высоты `y`, каждая — своим набором символов. Если `x` или `y` не положительны, функция не печатает ничего. Блок обрабатывает точное соответствие образцу вывода и служит основой для рейда `quadchecker`.

```
QuadA(5,3)    QuadB(5,3)
0---0         /***\
|  |         *  *
|  |         \***\
0---0
```

```
QuadC(5,3)  QuadD(5,3)  QuadE(5,3)
ABBBB      ABBBC       ABBBC
B  B       B  B        B  B
CBBBC      ABBBC       CBVBA
```

Отдельно разбираются граничные случаи: `x=1`, `y=1` (одна ячейка), вырожденные строка и столбец — именно на них видно, правильно ли описана логика углов и сторон.

Осваивается: вложенные циклы · логика границ фигуры · разбор граничных случаев · точность вывода

Quest 08 — Файлы, структуры и обработка ошибок · 5 упражнений

Блок о вводе-выводе: чтение файлов и стандартного ввода, аккуратная обработка ошибок, работа со структурами.

Что делает студент

- **boolean** — программа определяет, чётное или нечётное число аргументов ей передали, и печатает соответствующее сообщение.
- **point** — доработка функции `setPoint()` для работы со структурой через указатель; попутно — организация файлов проекта.
- **displayfile** — вывод содержимого файла с проверкой ошибок: аргумент отсутствует, аргументов слишком много, файл не открывается.
- **cat** — упрощённый аналог Unix-команды `cat`: чтение из стандартного ввода и из файлов, переданных аргументами, с обработкой отсутствующих файлов.
- **ztail** — упрощённый аналог `tail`: вывод последних символов файлов, продолжение работы при ошибке в одном из файлов, корректное форматирование при нескольких файлах.

Осваивается: файловый ввод-вывод · `stdin/stdout` · обработка ошибок без падения программы · структуры и указатели

Quest 09 — Функции высшего порядка · 8 упражнений

Блок о функциях как значениях: передача функции в функцию, функциональные литералы, настраиваемая логика обработки коллекций.

Что делает студент

- **foreach** — применение переданной функции к каждому элементу слайса целых.
- **map** — применение функции к каждому элементу и возврат нового слайса результатов.
- **any, countif** — проверка, удовлетворяет ли хотя бы один элемент условию, и подсчёт элементов, удовлетворяющих условию.
- **issorted** — проверка упорядоченности слайса с компаратором, заданным функцией `f`.
- **doop** — программа-калькулятор на трёх аргументах (значение, оператор, значение) с молчаливой обработкой некорректного ввода, деления на ноль и переполнения.
- **sortwordarr, advancedsortwordarr** — сортировка слайса строк по ASCII и сортировка по правилам, заданным переданной функцией сравнения.

Осваивается: функциональные типы и литералы · функции высшего порядка · компараторы · безопасная арифметика

Quest 11 — Связные списки · 15 упражнений

Первый блок о динамических структурах данных. Студент описывает структуры `NodeL / NodeI` и `List` и реализует полный набор операций над списком.

Что делает студент

- **Построение и разбор:** `ListPushBack`, `ListPushFront`, `ListClear` — добавление в конец и в начало, полная очистка списка.
- **Доступ и измерение:** `ListSize`, `ListLast`, `ListAt` — количество элементов, данные последнего элемента, узел по позиции (`nil` при некорректной позиции).
- **Преобразование:** `ListReverse` — разворот порядка узлов.
- **Обход с функцией:** `ListForEach`, `ListForEachIf` и функции `Add2_node`, `Subtract3_node` — применение функции ко всем узлам или только к узлам, удовлетворяющим условию.
- **Поиск и удаление:** `ListFind` с функцией сравнения `CompStr`, `ListRemoveIf` — удаление всех узлов с заданным значением.
- **Слияние и сортировка:** `ListMerge` (без создания новых элементов), `ListSort`, `SortListInsert` (вставка с сохранением порядка), `SortedListMerge` (слияние двух отсортированных списков).

Осваивается: объявление структур · указатели и связи между узлами · обход и модификация списка · алгоритмы сортировки и слияния

Quest 12 — Бинарные деревья · 12 упражнений

Завершающий блок: структура `TreeNode` со ссылками на левого, правого потомка и родителя, свойства бинарного дерева поиска, все основные обходы.

Что делает студент

- **`btreeinsertdata`** — вставка данных с сохранением свойств дерева поиска.
- **`btreeapplyinorder`, `btreeapplypostorder`, `btreeapplypreorder`, `btreeapplybylevel`** — применение функции к элементам при обходе в четырёх порядках, включая обход по уровням.
- **`btreeearchitem`** — поиск элемента с возвратом узла или `nil`.
- **`btreelevelcount`** — вычисление высоты дерева.
- **`btreeisbinary`** — проверка, что дерево удовлетворяет правилам бинарного дерева поиска.
- **`btreemin`, `btreemax`** — поиск узлов с минимальным и максимальным значением.
- **`btreetransplant`, `btreedeletenode`** — замена поддеревя и удаление узла с сохранением корректности структуры.

Осваивается: древовидные структуры · рекурсивные обходы · свойства BST · вставка и удаление узлов · функциональная обработка узлов

5. Checkpoint: как проверяется освоение

Чекпоинты проходят на неделях 2, 4, 6 и 8 — в пятницу и субботу, в аудитории, очно и индивидуально. В каждый день работают два потока: первый с 08:00 до 10:30, второй с 10:30 до 13:00; на решение отводится 2 часа 30 минут. Задачи опираются на материал пройденных недель, но формулируются иначе, чем в квестах: готовое решение из квеста не подходит, нужно применить навык к новой постановке.

Пример задачи уровня «функция»

Написать функцию, возвращающую медиану пяти целых аргументов.

```
func Abort(a, b, c, d, e int) int {
}
```

Проверочная программа и её вывод:

```
package main

import (
    "fmt"
    "piscine"
)

func main() {
    middle := piscine.Abort(2, 3, 8, 5, 7)
    fmt.Println(middle)
}

$ go run .
5
$
```

Пример задачи уровня «программа»

Написать программу, решающую sudoku. Корректное sudoku имеет единственное решение; в сдачу входят все файлы, необходимые для запуска программы.

```
$ go run . ".96.4...1" "1...6...4" "5.481.39." "..795..43" ".3..8...." \
    "4.5.23.18" ".1.63..59" ".59.7.83." "..359...7" | cat -e
3 9 6 2 4 5 7 8 1$
1 7 8 3 6 9 5 2 4$
5 2 4 8 1 7 3 9 6$
2 8 7 9 5 1 6 4 3$
9 3 1 4 8 6 2 7 5$
4 6 5 7 2 3 9 1 8$
7 1 2 6 3 8 4 5 9$
6 5 9 1 7 4 8 3 2$
8 4 3 5 9 2 1 6 7$
$
```

Некорректный ввод и нерешаемое sudoku обрабатываются единообразно:

```
$ go run . 1 2 3 4 | cat -e
Error$
$ go run . | cat -e
Error$
```

Такая задача требует одновременно рекурсии с возвратом (как в «восьми ферзях»), работы с аргументами командной строки, валидации ввода и точного формата вывода — то есть материала нескольких квестов сразу.

6. Raid: групповая работа

Рейды выдаются на неделях 2, 4 и 6 и выполняются онлайн на выходных — в субботу и воскресенье. Задание одно на группу, результат защищается целиком и уже на следующей неделе — очно, перед менторами и преподавателями. Важен не только работающий код, но и то, как участники разделили работу, состыковали её части и способны ли объяснить решение целиком.

Пример рейда: quadchecker

Рейд построен на функциях `quad`. Нужна программа `quadchecker`, которая принимает строку и определяет, какой `quad` её нарисовал, вместе с размерами фигуры.

- Если аргумент не является результатом `quad`-функции, программа печатает `Not a quad function`.
- Все ответы завершаются переводом строки (`'\n'`).
- Если подходит несколько функций, выводятся все — по алфавиту, через `||`.

Структура сдачи:

```
$ tree .
quadchecker
├── go.mod
└── main.go
```

Примеры работы:

```
$ ./quadA 3 3 | go run .
[quadA] [3] [3]

$ ./quadC 1 1 | go run .
[quadC] [1] [1] || [quadD] [1] [1] || [quadE] [1] [1]

$ ./quadE 1 2 | go run .
[quadC] [1] [2] || [quadE] [1] [2]

$ echo 0 0 | go run .
Not a quad function
```

Задача намеренно неоднозначна: на малых размерах разные `quad`-функции дают одинаковый вывод, поэтому группе придется договариваться о том, как устроено сравнение, кто отвечает за разбор входа, кто — за формирование ответа, и как это стыкуется в одну программу.

Чему учит рейд

- Разделять задачу на части, которые можно вести параллельно.
- Читать и принимать чужой код, договариваться об интерфейсах между частями и работать в одном репозитории удалённо.
- Держать срок: результат нужен к защите на следующей неделе, в назначенное менторами время.
- Объяснять решение — на защите перед менторами и преподавателями спрашивают любого участника группы.

7. Требования к работам и итоговые компетенции

Общие требования

- **Полнота сдачи.** В репозиторий отправляются все файлы, необходимые для запуска программы (включая `go.mod`, где он требуется).
- **Точность вывода.** Вывод сверяется буквально, вплоть до пробелов и переводов строк; для контроля используется `cat -e`.
- **Соблюдение имён.** Имена файлов, функций и пакетов заданы условием и не меняются.
- **Ограничения по инструментам.** В части заданий запрещены отдельные конструкции (например, `for` в рекурсивных задачах, `append` или `make` в задачах на слайсы, приведение типов в задачах на вывод алфавита) — ограничение и есть предмет обучения.
- **Корректное поведение на некорректном вводе.** Программа не падает: она возвращает нулевое значение, печатает `Error` или не печатает ничего — в зависимости от условия.
- **Эффективность.** Ряд заданий проверяется на время: решение «в лоб» уходит в таймаут и не принимается.

Итоговые компетенции

Область	Что студент умеет по итогам бассейна
Рабочая среда	Работать в командной строке Unix, вести проект в Git, писать и отлаживать shell-скрипты, автоматизировать рутинные операции с файлами и данными.
Основы Go	Писать функции и программы, управлять потоком исполнения, работать с типами, преобразованиями и форматированием вывода.
Память и указатели	Осознанно использовать указатели, изменять данные по ссылке, понимать разницу между значением и адресом.
Алгоритмы	Реализовывать итеративные и рекурсивные решения, оценивать эффективность, применять перебор с возвратом.
Строки и данные	Обрабатывать строки и руны, валидировать ввод, разбирать и собирать текст, конвертировать числа между системами счисления, работать с JSON.
Структуры данных	Проектировать и использовать связанные списки и бинарные деревья, реализовывать обходы, поиск, вставку, удаление, сортировку и слияние.
Функциональные приёмы	Передавать функции как аргументы, писать функции высшего порядка и настраиваемые компараторы.
Ввод-вывод и ошибки	Читать файлы и стандартный ввод, обрабатывать ошибки без остановки программы, работать с аргументами и флагами.
Рабочие навыки	Самостоятельно разбирать техническое задание, искать информацию, работать в команде, укладываться в срок и защищать результат.